

```
require("tidyverse")
require("MASS")
```

Simulations to explore the efficiency of stepwise regression procedures for variable selection in multiple linear regression models

When building a linear regression model we often need to decide which explanatory variables to include in the model. As discussed in class, this can be a difficult decision and will depend on the purpose of the model. You may have encountered a technique called step-wise regression that is sometimes used to select which variables to include in a regression model. Step-wise regression is an automated procedure where at each step the inclusion of a variable is assessed according to some predefined criterion. The order in which variables are considered and the criterion for inclusion can vary across different step-wise regression methods. Forward step-wise regression starts with an intercept only model and adds variables step by step. Backwards step-wise regression starts with the full model, containing all explanatory variables and deletes variables step by step. Other methods use a combination of both forwards and backwards selection and some consider all possible combinations of subsets of variables.

In this worksheet we will explore the method of backwards step-wise regression using the `stepAIC()` function from the MASS package. The measure used to decide whether to retain a variable in the model is the AIC (Akaike's Information Criterion) value which is a penalised measure of model fit given by:

$AIC = 2k - 2\ln(L)$ where k is the number of parameters in the model and L is the maximum value of the likelihood function of the model.

AIC takes into account the number of explanatory variables as well as the fit of the model. AIC can be used to compare models where the likelihoods are computed on the same data: the model with the lower AIC is considered a better model. In the context of backwards step-wise regression, at each step a variable is dropped and the AIC is calculated for each model, if $AIC(\text{larger model}) < AIC(\text{smaller model})$ then the variable is retained.

Step-wise regression is not recommended for variable selection for several reasons (taken from Harrell 2001) including:

- R^2 values are biased high
- the standard errors of the parameter estimates are too small
- the confidence intervals around the parameter estimates are too narrow
- p-values are too low, (due to multiple comparisons)
- parameter estimates are biased away from 0

The problems listed above stem from using data obtained from a random sample to specify a model. We expect variability in our sample estimates for the population regression coefficients. By only retaining coefficients that have reached a specified threshold in one particular sample will bias the selection procedure in the ways described above. See the step-wise regression section on Canvas for further discussion of the problems associated with this technique and for other simulation studies exploring this topic.

In this worksheet we will investigate the results obtained using the `stepAIC()` function for model selection by running simulations where the true nature of the relationship between the explanatory variables and the response variable is known.

Single application of `stepAIC()`

We begin by creating a single data set with 20 predictors, 5 of which are linearly related to the outcome Y and 15 of which are noise. An important consideration in regression analysis is the signal to noise ratio

i.e. the magnitude of the effect size (the β coefficients) to the variance. To start, set the β coefficients that are linearly related to the outcome to 0.5 and set the variance of the error term to 1. We create a data set with 500 observations.

Data Generation

Let k represent the number of explanatory variables X_j $j = 1 \dots k$

Each explanatory variable is drawn from a Standard Normal distribution $X_j \sim N(0, 1)$.

Let k_ln represent the number of explanatory variables that are linearly related to the response variable Y .
Let B_mag represent the magnitude of the population β coefficients that are linearly related to the response Y .

The i_{th} observation of the response variable is defined to be

$$Y_i = \beta_0 + \beta_1 X_{i1} + \dots + \beta_k X_{ik} + e_i$$

where $e_i \sim N(0, 1)$

```
set.seed(2) # so can reproduce results
k = 20 # 20 explanatory variables
k_ln = 5 # 5 explanatory variables are linearly related to Y
B_mag = 0.5 # the magnitude of the explanatory variables that are linearly related to Y
n = 500 # number of observations in data set

# create a vector containing the population beta coefficients
B <- c(rep(B_mag, k_ln), rep(0, k - k_ln))
X <- matrix(rnorm(n * k), nrow = n) # create the explanatory variables
Y <- X %*% B + matrix(rnorm(n), nrow = n) # create the response variable

# combine the response and explanatory variables in a single dataframe
DF <- cbind(Y, X)
DF <- as.data.frame(DF)

# assign column names
colnames(DF) <- c("Y", "X1", "X2", "X3", "X4", "X5", "X6", "X7", "X8", "X9", "X10", "X11",
                  "X12", "X13", "X14", "X15", "X16", "X17", "X18", "X19", "X20")
```

Stepwise regression

Next we perform step-wise regression to choose a model. Using the `stepAIC()` function, a variable is retained if $AIC(\text{large model}) < AIC(\text{small model})$, this is equivalent to keeping larger model if the p-value associated with the dropped variable is less than 0.15

```
# Fit linear regression model using all the predictors
full_model <- lm(Y ~ ., data = DF)

# the results of the process are stored in step.model
step.model <- stepAIC(full_model, direction = "backward")
```

Let's examine the model selected using `stepAIC()`

```
summary(step.model)
```

```
##
## Call:
## lm(formula = Y ~ X1 + X2 + X3 + X4 + X5 + X12 + X17, data = DF)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.7285 -0.6307 -0.0416  0.6102  2.9973
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.04845     0.04642   1.044  0.2971
## X1           0.53585     0.04463  12.006 <2e-16 ***
## X2           0.44211     0.04612   9.585 <2e-16 ***
## X3           0.48978     0.04778  10.251 <2e-16 ***
## X4           0.50391     0.04504  11.189 <2e-16 ***
## X5           0.47556     0.04431  10.733 <2e-16 ***
## X12          0.08194     0.04810   1.703  0.0891 .
## X17          0.07035     0.04515   1.558  0.1198
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.025 on 492 degrees of freedom
## Multiple R-squared:  0.5503, Adjusted R-squared:  0.5439
## F-statistic: 86.03 on 7 and 492 DF,  p-value: < 2.2e-16
```

The model has correctly retained variables $X_1 \dots X_5$ but has incorrectly included variables X_{12} and X_{17} . If we examine the estimated coefficients, we see that the estimates for the β coefficients ($\beta_1, \dots, \beta_5$) are close to their true value of 0.5 and the estimate for β_{12} is 0.08194 and the estimate for β_{17} is 0.07035. If we let $T1$ denote the number of variables that were retained incorrectly and $T2$ represent the number of variables that were omitted incorrectly then in this example, $T1 = 2$ and $T2 = 0$.

Multiple data sets to examine the performance of stepAIC()

In the next section of this worksheet we scale up the number of data sets we create and fit a linear regression model to using the stepAIC() function. The aim is to examine the number of variables that were retained incorrectly ($T1$) and the number of variables that were omitted incorrectly ($T2$) over many repetitions. In addition to increasing the number of repetitions, we will explore the impact of varying:

- (a) the number of variables that are linearly related to the outcome Y
- (b) the strength of the relationship between the explanatory variables and the outcome (the signal to noise ratio).

Let k represent the number of explanatory variables X_j $j = 1 \dots k$

Each explanatory variable is drawn from a Standard Normal distribution $X_i \sim N(0, 1)$.

Let k_ln represent the number of explanatory variables that are linearly related to the response variable Y .

Let B_mag represent the magnitude of the population β coefficients that are linearly related to the response Y .

Let n represent the number of observations in the data set

The i th observation of the response variable is defined to be

$$Y_i = \beta_0 + \beta_1 X_{i1} + \dots + \beta_k X_{ik} + e_i$$

where $e_i \sim N(0, 1)$

Let *reps* represent the number of simulations for each combination of parameters.

For our analysis we set:

```
k = 20    # 20 explanatory variables

# we explore the cases where there are 0, 1, 5 and 10 variables that are
# linearly related to the outcome
k_ln = c(0, 1, 5, 10)

# we examine the effect of changing the strength of the beta coefficients that
# are linearly related to the response Y.
B_mag = c(0.01, 0.05, 0.1, 0.5, 1, 2)
n = 500  # 500 observations in each dataset
reps = 100 # 100 repetitions of each case
```

We generate 100 data sets with 500 observations in each data set for each combination of parameters resulting in $4 \times 6 \times 100 = 2400$ simulations

Note that varying the size of the non zero β coefficients changes the signal: noise ratio from 0.01 : 2

We record the results of the step-wise regression in a data frame named **Results**. Each row in the **Results** data frame represents the results from a step-wise regression for a particular replicate, for a particular combination of parameters.

Data Generation and stepAIC()

```
# set up data frame to record results
# Rep records the number of the simulation/replicate 1:100
# N_lin records the number of explanatory variables that are linearly related to the response
# beta_mag records the strength of the non -zero beta coefficients
# T1 records the number of variables that were retained incorrectly
# T2 records the number of variables that were omitted incorrectly
Results <- expand.grid(Rep = 1:reps,
                      No_lin = k_ln,
                      beta_mag = B_mag)

Results$T1 <- rep(0, reps * length(k_ln) * length(B_mag))
Results$T2 <- rep(0, reps * length(k_ln) * length(B_mag))

# Set up a list to record the final model chosen by stepAIC for each simulation
list_models <- vector(mode = "list", length = reps * length(k_ln) * length(B_mag))

# For each combination of parameters, create the population beta coefficients
for(j in k_ln) {
  for(b in B_mag){

    B <- if(j == 0) rep(0, k) else c(rep(b, j), rep(0, k - j))

    # For each simulation and combination of parameters, create the explanatory variables
    # and response variable and then fit a linear regression model
```

```

for(m in 1:reps) {
# create matrix containing explanatory variables, randomly sampled from a
# standard normal distribution
  X <- matrix(rnorm(n*k), nrow=n)
# create response variable Y = BX +Error
# (Error sampled randomly from a standard normal distribution)
  Y <- X%*%B + matrix(rnorm(n),nrow=n)
  DF <- cbind(Y,X) # create dataframe
  DF<- as.data.frame(DF)
  # name columns
  colnames(DF)<-c("Y","X1","X2","X3","X4","X5","X6","X7","X8","X9","X10","X11","X12",
                 "X13","X14","X15","X16","X17","X18","X19","X20")
  # run linear model
  full_model <- lm(Y ~ .,data = DF)

  # perform stepwise regression to choose a model
  step.model<-stepAIC(full_model, trace=FALSE)

# To evaluate the performance of stepwise regression we would like to keep
# track of the number of variables that were retained incorrectly (T1) and the
# number of variables that were omitted incorrectly (T2). The next section of
# code extracts and records this information.

# Extract list of retained variables from model chosen using stepAIC
# The terms() function extracts terms associated with step.model
# From this output the term labels are extracted using the attr() function
  Retained<-attr(terms(step.model),"term.labels")
  Retained<-as.numeric(gsub("X", "", Retained))

# Create a vector of length k with 1 in the position of a retained variable
# and 0 in the position of a dropped variable.
  Vector_step <-rep(0,k)
  Vector_step <-replace(Vector_step, Retained, 1)

# Create a vector of length k with 1 in the position of a variable that has a linear relationship with
  Vector_B <- B
  Vector_B[Vector_B > 0] <- 1

# Count where the two vectors do not match for variables where there is a linear relationship
# and where the two vectors do not match for variables where there is not a linear relationship
# t1 counts variables incorrectly retained
# t2 counts variables incorrectly omitted
  ifelse(j==0, t2<-0, t2 <-length(which(Vector_B[1:j] != Vector_step[1:j])))
  t1<-length(which(Vector_B[(j+1):k] != Vector_step[(j+1):k]))

# identify the correct row in which to store the results
  h <- which(Results$Rep == m & Results$No_lin == j & Results$beta_mag == b)
#collect results in dataframe
  Results[h,4]<-t1
  Results[h,5]<-t2

```

```

    # collect chosen models in a list
    list_models[[h]]<-step.model

  }
}

```

Lets examine the first few results:

```
head(Results)
```

```

##   Rep No_lin beta_mag T1 T2
## 1    1      0    0.01  8  0
## 2    2      0    0.01  3  0
## 3    3      0    0.01  5  0
## 4    4      0    0.01  6  0
## 5    5      0    0.01  4  0
## 6    6      0    0.01  6  0

```

The output shows that for the first simulation where no explanatory variables were linearly related to the outcome, eight variables were retained incorrectly in the final model and no variables were incorrectly omitted (this must be the case since no variables were linearly related to the outcome). Note that when `No_lin = 0`, the value of `beta_mag` is irrelevant since all coefficients have the value 0. We will tidy this up at the end.

Summarising the Results

To get an overview of the performance of the `stepAIC()` function we can summarise the results by averaging the results over all the simulations for each parameter combination. Specifically, we calculate (a) the average number of variables retained incorrectly over all simulations for each combination of parameters (b) the average number of variables omitted incorrectly for each combination of parameters.

```

# set up summary table to record the results
summary_table<-expand.grid(No_lin = k_ln,
                           beta_mag = B_mag)
# create a new variable ave_t1 to record the average percentage of incorrectly
# retained variables
summary_table$ave_t1<-rep(0,length(summary_table))
# create a new variable ave_t2 to record the average percentage of incorrectly
# omitted variables
summary_table$ave_t2<-rep(0,length(summary_table))

# average over each combination of parameters
for(b in B_mag) {
  for(j in k_ln){

    #select the results for each combination of parameters
    t<- filter(Results, Results$No_lin == j & Results$beta_mag == b)
    # identify the correct row of the summary table to store the results
    h <- which(summary_table$No_lin == j & summary_table$beta_mag == b)
    # calculate and record the averages
    summary_table[h,3]<-100*sum(t$T1)/(reps*k)
  }
}

```

```
summary_table[h,4]<-100*sum(t$T2)/(reps*k)

}}

print(summary_table)
```

```
##      No_lin beta_mag ave_t1 ave_t2
## 1         0      0.01  17.20   0.00
## 2         1      0.01  16.75   4.20
## 3         5      0.01  12.85  20.75
## 4        10      0.01   8.70  41.90
## 5         0      0.05  15.90   0.00
## 6         1      0.05  16.10   2.80
## 7         5      0.05  10.85  15.35
## 8        10      0.05   8.40  30.40
## 9         0      0.10  16.30   0.00
## 10        1      0.10  14.65   0.70
## 11         5      0.10  14.20   5.25
## 12        10      0.10   8.90  10.90
## 13         0      0.50  16.60   0.00
## 14         1      0.50  16.55   0.00
## 15         5      0.50  13.20   0.00
## 16        10      0.50   9.00   0.00
## 17         0      1.00  16.95   0.00
## 18         1      1.00  15.10   0.00
## 19         5      1.00  13.20   0.00
## 20        10      1.00   7.55   0.00
## 21         0      2.00  16.55   0.00
## 22         1      2.00  15.30   0.00
## 23         5      2.00  12.55   0.00
## 24        10      2.00   8.40   0.00
```

Recall that for the simulations where no explanatory variables were linearly related to the response variable ($\text{No_lin} = 0$), the value of beta_mag is irrelevant since all coefficients have the value 0. Let's average all the results where $\text{No_lin} = 0$ and put the results into the first row.

```
# select the rows where No_lin = 0
t<- filter(Results, Results$No_lin == 0)
# record the average percentage in the first row of the table
summary_table[1,3]<-100*sum(t$T1)/(reps*k*6)
summary_table[1,4]<-100*sum(t$T2)/(reps*k*6)
#remove other rows from summary table where No_lin = 0
summary_table<-summary_table[-c(5,9,13,17,21),]

# replace the value of beta_mag with 0
summary_table[1,2]<-0
print(summary_table)
```

```
##      No_lin beta_mag ave_t1 ave_t2
## 1         0      0.00 16.58333   0.00
```

## 2	1	0.01	16.75000	4.20
## 3	5	0.01	12.85000	20.75
## 4	10	0.01	8.70000	41.90
## 6	1	0.05	16.10000	2.80
## 7	5	0.05	10.85000	15.35
## 8	10	0.05	8.40000	30.40
## 10	1	0.10	14.65000	0.70
## 11	5	0.10	14.20000	5.25
## 12	10	0.10	8.90000	10.90
## 14	1	0.50	16.55000	0.00
## 15	5	0.50	13.20000	0.00
## 16	10	0.50	9.00000	0.00
## 18	1	1.00	15.10000	0.00
## 19	5	1.00	13.20000	0.00
## 20	10	1.00	7.55000	0.00
## 22	1	2.00	15.30000	0.00
## 23	5	2.00	12.55000	0.00
## 24	10	2.00	8.40000	0.00

Please write down a few sentences to summarise the results from these simulations.